

任务指导 · 智能多工具助手

这是带你上手的教程。配套还有两份：`README.md` 是命令、接口、评分表的速查，`docs/背景知识.md` 是概念原理的详解。

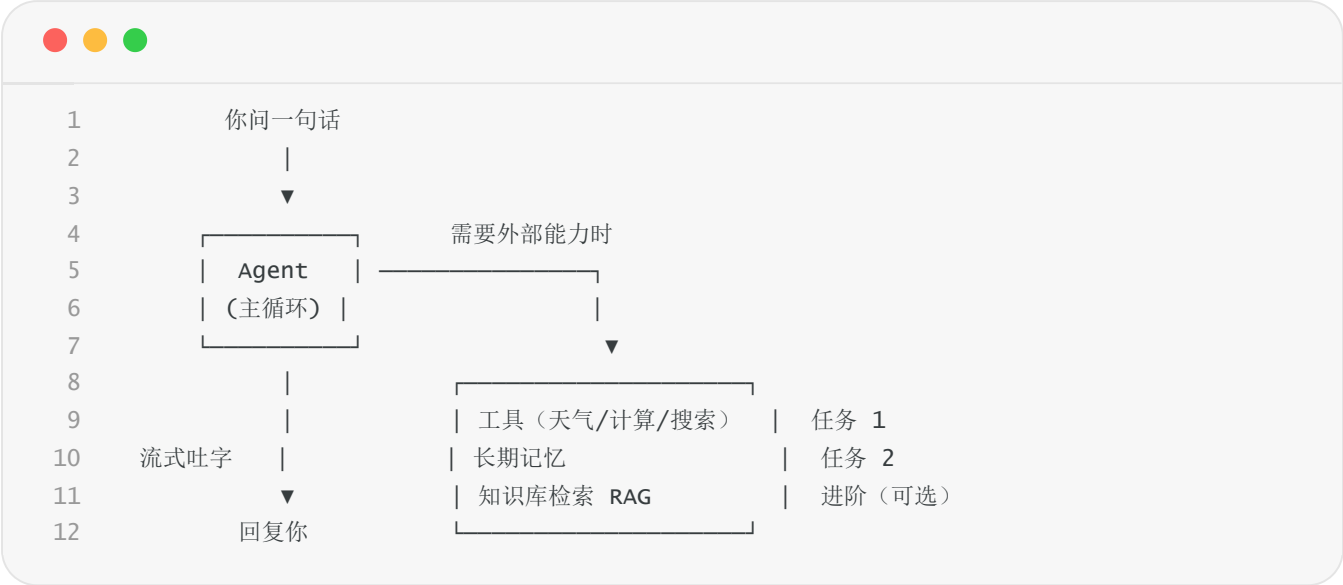
先说两件事：

- 工具、记忆 这些概念的「是什么、为什么」都抽到了配套的 `docs/背景知识.md`。
- 两个任务互不影响。顺序建议见第 6 节。

1. 你要做什么

补全一个 AI 助手，让它会查天气、搜网络，还能跨会话记住你。

目前已经把引擎和两个完整范例写好了。你要做的，是照着范例把另外几个能力模块里的 `TODO` 填上。不要求写测试。



每个能力都是可插拔的。写好、打开开关，助手就多一项本领；不写，它就自动隐藏，不会影响别的部分。

2. 五分钟跑起来

2.1 安装与配置

```
1 pip install -r requirements.txt
2 cp .env.example .env      # 填入 OpenAI 兼容服务的 key / base_url / 模型名
```

`.env` 一共四项：`OPENAI_API_KEY`、`OPENAI_BASE_URL`、`CHAT_MODEL`、`EMBEDDING_MODEL`。默认填的是 DeepSeek 的示例值，换成你自己的服务就行。

可以去火山引擎申请免费的 API，额度为 50W tokens。申请教程可以参考这个视频https://www.bilibili.com/video/BV191fbBnEPa/?spm_id_from=333.337.search-card.all.click&vd_source=2244fc884e12b7901e69474454120181

2.2 动手前先跑两个自检

自检	命令	应该看到	说明
跑测试	<code>pytest -v</code>	全部通过	测试走的是假客户端（ <code>tests/fakes.py</code> ），没配 key 也能跑。这步绿了，说明环境没问题。
验 key	<code>python main.py</code>	助手能回话	启动时会打印 <code>已启用工具: [...]</code> ，这是你看到的第一张能力清单。

2.3 起后端，配网页界面

```
1 uvicorn server:app --reload
```

浏览器打开 <http://localhost:8000>，界面分三栏：

- 左边会话栏：多个对话，可以新建、切换、重命名、删除。历史会自动保存，重启服务还在。
- 中间对话区：和助手流式聊天。
- 右边仪表栏：实时显示助手调了哪个工具、引用了哪些来源、记住了你什么、启用了哪些能力。

右边这栏很重要。后面验证每个任务有没有生效，基本都靠它。

两种运行方式怎么选：想快速验证某个函数，`python main.py` 最省事；想看工具、来源、记忆的完整效果，用网页。

3. 项目长什么样

3.1 完整结构



1	homework/	
2	— main.py	命令行入口，单条会话、历史不落盘
3	— server.py	FastAPI 后端：SSE 接口 + 多会话管理，顺带托管前端
4	— factory.py	装配 Agent：注册工具、按开关决定用哪种长期记忆
5	— agent.py	主循环：function calling + 事件流
6	— llm.py	大模型客户端：封装 chat、流式、embedding
7	— config.py	读 .env，提供 USER_ID 等配置
8	— requirements.txt	
9	— .env / .env.example	模型服务的 key、base_url、模型名
10		
11	— tools/	工具
12	— base.py	Tool 抽象基类
13	— registry.py	工具注册表：收集 schema、按名分发、异常兜底
14	— weather.py	★ 范例：天气工具（mock 数据）
15	— calculator.py	★ 范例：计算器工具
16	— web_search.py	☆ 任务 1：网络搜索（待你实现）
17	— knowledge_base.py	☆ 进阶（可选）：知识库检索工具（RAG）
18		
19	— rag/	检索增强（进阶内容，不做不影响）
20	— vector_store.py	☆ 进阶：向量库（numpy 余弦检索）+ 共享实例 STORE
21	— ingest.py	☆ 进阶：文档切块、嵌入、入库
22		
23	— memory/	记忆
24	— base.py	长期记忆抽象接口 + NoOp 兜底实现
25	— short_term.py	★ 范例：短期记忆（多轮历史 + 滚动摘要）
26	— long_term.py	☆ 任务 2：长期记忆，跨会话（待你实现）
27	— conversation_store.py	对话持久化，老师已写好，不用动
28		
29	— frontend/	
30	— index.html	三栏网页界面
31		
32	— tests/	测试
33	— fakes.py	假 LLM、假工具
34	— test_weather.py	★ 范例：工具测试
35	— test_short_term.py	★ 范例：记忆测试
36	— test_*.py	引擎自带的测试
37		
38	— data/	
39	— docs/	知识库语料（知简-*.md，虚构产品）
40	— conversations/	对话历史
41	— memory_store.json	长期记忆，任务 2 做完后生成

```
42 |  
43 └─ docs/  
44   └─ 任务指导.md          你正在看的这份（怎么做）  
45   └─ 背景知识.md          工具/记忆 的原理详解（为什么）  
46   └─ rag-verification.md  RAG 验证清单（进阶可选）
```

3.2 数据流

记住一句话就够：`agent.chat_stream()` 把一轮对话拆成一串事件，挨个抛出来。`token` 是模型在吐字，`tool_call` 是它调了某个工具，`sources` 是这次引用的文档来源，`done` 是这一轮收尾。右边仪表栏显示的工具调用、引用来源，就是前端在实时接这些事件画出来的，所以你后面验证每个任务，本质都是在看这串事件对不对。

流式那些脏活都封装在 `agent.py` 和 `llm.py` 里了，你写工具和记忆时面对的全是普通同步函数，碰不到流式。

3.3 最该记住的一个模型：开关

整个项目能做到任务互不影响，靠的是每个能力都有一个开关：

能力	开关在哪	关着时
工具（如 <code>web_search</code> ）	工具的 <code>is_available()</code> 返回 <code>False</code>	工具对模型不可见，模型根本不会调它
长期记忆	<code>memory/long_term.py</code> 顶部的 <code>ENABLED = False</code>	<code>factory</code> 改用 <code>NoOpLongTermMemory</code> ，什么都不做

由此带来一个最常见的困惑：代码明明写了，效果却不出现。十有八九是开关没打开。

确认开关状态有两个办法。一是看网页右边仪表栏的「已启用能力」，二是直接查接口：

```
1 curl http://localhost:8000/api/capabilities  
2 # {"tools": [...], "rag_enabled": true/false, "long_term_enabled": true/false}
```

`python main.py` 启动那行也会打印当前启用的工具。

4. 先读范例

照着范例写，是这个作业的正确做法。动手前花十分钟把对应范例看懂，能省下后面大把时间。

4.1 一个工具长什么样：tools/weather.py

一个工具就是继承 `Tool`，填好四样东西：

- `name`：function calling 里的唯一名字。
- `description`：给模型看的说明，模型靠它判断什么时候该调这个工具。写得含糊，模型就不会用。
- `parameters`：参数的 JSON Schema。
- `run(...)`：真正干活的地方，返回一个字符串。

`weather.py` 用的是内置 mock 数据，所以没 key、不联网也能跑通整条链路。剩下的事，比如转成 OpenAI 格式（`openai_schema()`）、注册、分发、异常兜底，都在 `tools/base.py` 和 `tools/registry.py` 里替你做好了。

工具是怎么被模型「点」到、`run()` 的返回又怎么回到对话里的，这套 function calling 的循环机制，`docs/背景知识.md` 第 1 节拆成五步讲得很细。没接触过的先去看一眼，对理解两个任务都有帮助。

4.2 一个记忆模块长什么样：memory/short_term.py

这是你写长期记忆的对照样板。重点看它三件事怎么做的：内容存在哪（`history` 列表加一段滚动 `summary`），什么时候调用 LLM 加工（历史太长时压成摘要），怎么把内容注入对话（`get_context()` 拼出 system 消息）。

4.3 范例对照速查

你要写	照着读
一个新工具（任务 1）	<code>tools/weather.py</code> ， <code>calculator.py</code> 是第二个例子
工具的测试	<code>tests/test_weather.py</code>
一个记忆模块（任务 2）	<code>memory/short_term.py</code> 加 <code>tests/test_short_term.py</code>

5. 两个任务

5.1 任务 1 · 工具调用 web_search

是什么：一个网络搜索工具，模型需要实时信息时调用。文件是 `tools/web_search.py`。

不清楚「工具」「function calling」是怎么回事？`docs/背景知识.md` 第 1 节讲了：为什么大模型需要工具、为什么模型只「点单」不执行代码、`description` 为什么是关键。

在流程里的位置：和 `weather` 走的是同一套工具循环（见 4.1），本质就是再加一个工具。

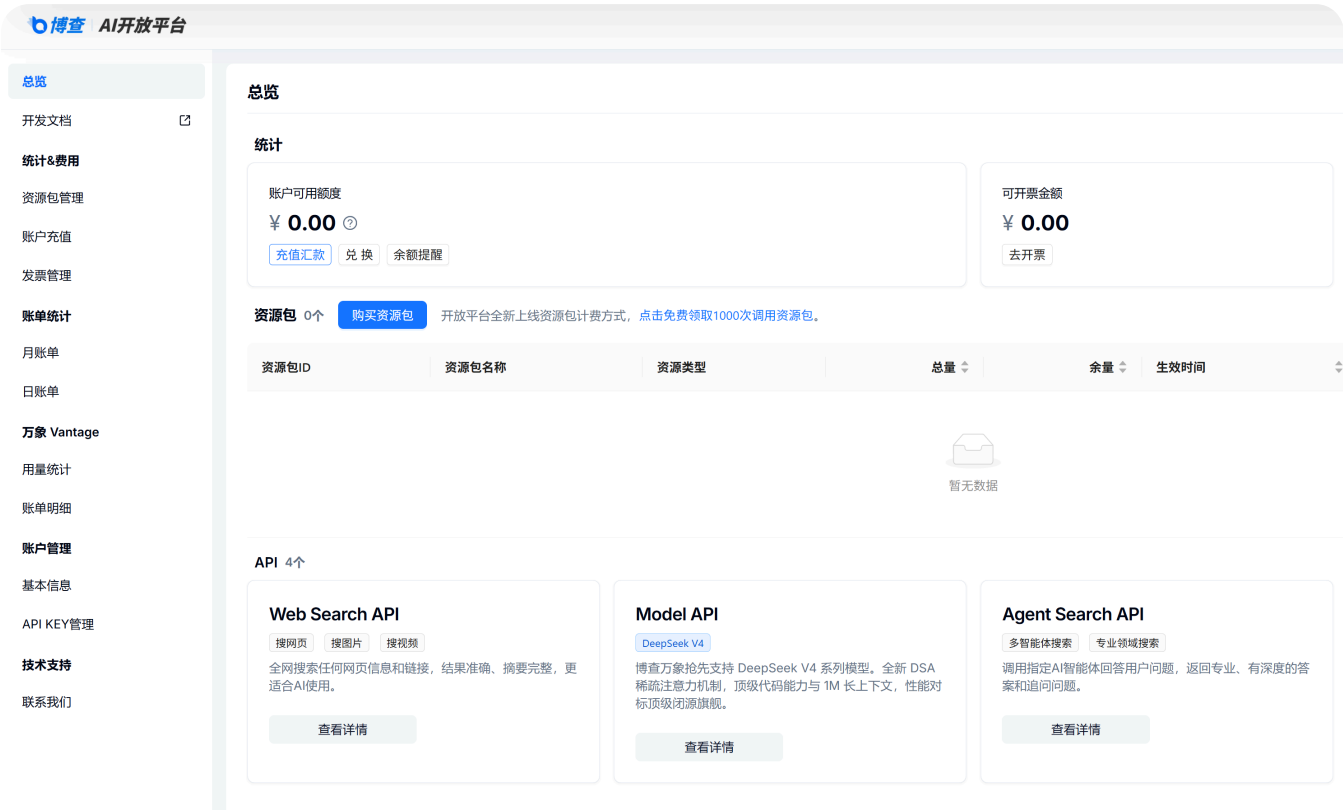
思路：这是两个任务里最简单的，建议第一个做，先把工具循环、开关、写测试这套流程跑顺。照 `weather.py` 的样子，`run()` 返回一个字符串就行。先用 mock 数据把链路跑通，确认工具能被调到，再去考虑接真实搜索 API。写完别忘了改开关，骨架里 `is_available()` 的 `TODO` 已经提示了。

最小验证：起后端，问助手「搜一下今天的新闻」，右边仪表栏会出现调用 `web_search` 的记录（用 `python main.py` 则会打印 `[调用工具 web_search ...]`）。`/api/capabilities` 的 `tools` 里也应该能看到 `web_search`。

易踩坑：

- 实现了 `run()`，却忘了把 `is_available()` 改成 `True`，结果工具被隐藏，模型永远不调。
- `description` 写得太笼统（比如只写「搜索」），模型判断不出什么时候该用。描述要让模型一看就知道适用什么场景。

关于 API：可以使用博查平台的 API，有 1000 次免费额度。<https://open.bochaai.com/>



5.2 任务 2 · 长期记忆

是什么：跨会话记住用户的稳定事实和偏好，比如「我叫小明」「我是 Python 工程师」，落盘到 `data/memory_store.json`。文件是 `memory/long_term.py`。

不熟悉「大模型为什么会失忆」「记忆到底是什么」？`docs/背景知识.md` 第 2 节讲清了：模型为什么无状态、两层记忆分别治哪种「忘」、为什么长期记忆要按全局 `USER_ID` 走，以及任何记忆都在做的「写」和「读」两件事。

在流程里的位置：`agent.chat_stream()` 每轮开头调 `recall(user_msg)`，把相关事实注入上下文；每轮结尾调 `remember(user_msg, reply)`，抽取并存下事实。接口（`recall`、`remember`、`all_facts`）定义在 `memory/base.py`，没实现时由 `NoOpLongTermMemory` 兜底。

思路：对照 `short_term.py` 看一个记忆模块怎么存、何时调 LLM、怎么注入，长期记忆是同一套思路，只是换成跨会话的事实。四个待实现的方法各管一摊：`_load` 和 `_save` 管落盘，`recall` 管召回，`remember` 管用 LLM 从对话里抽稳定事实。先用骨架建议的最简单策略（关键词召回、全量注入）跑通，向量召回留作进阶。

有一点要特别注意：记忆是全局的。它按固定的 `USER_ID`（见 `config.py`）存，所有对话窗口共享同一份，和每个窗口的 `session_id` 是分开的。这正是换个窗口也能记得的原因。写完把模块顶部的 `ENABLED` 改成 `True`，`factory` 会自动启用。

最小验证：起后端，在 A 窗口告诉它「我叫小明」，再新建一个 B 窗口问「我叫什么」。它答得出来，就成了，因为长期记忆是全局共享的。也可以 `curl http://localhost:8000/api/memory/任意id` 看 `facts` 列表，或者看右边仪表栏的「记住了什么」。

易踩坑：

- `_save()` 直接覆盖文件，会把别的 user 的数据冲掉。正确做法是先读出整体 dict，只更新自己这一项，再写回。
- 把记忆按 `session_id` 存，换个窗口就忘了，跨会话的效果也没了。它要按全局 `USER_ID` 存。
- 同一条事实反复存进去，记得去重。

6. 推荐的做题顺序

- 
- 1 任务 1（工具，最简单，先尝试工具循环、开关）
 - 2 → 任务 2（长期记忆，有 `short_term` 直接对照）

不过两个任务互不影响，做完一个就能在界面里看到效果，不必等全做完。

7. 几个常见疑问

- 我明明写了，怎么没生效，界面也没变化？
 - 绝大多数情况是开关没打开。查一下 `/api/capabilities`（或右边仪表栏），确认你的能力是 `true`。工具看 `is_available()`，长期记忆看 `ENABLED`。
- SSE、流式这些要我自己处理吗？
 - 不用。引擎把流式封装在 `agent.py` 和 `llm.py` 里了，你写的 `run()` 和记忆方法都是普通同步函数。

8. 优化方向

RAG（本地知识库检索）是一个完整的进阶方向。仓库里保留了它的全套材料：`rag/` 与 `tools/knowledge_base.py` 骨架、`data/docs/` 语料、`docs/rag-verification.md` 验收清单——但实现思路和代码都留给你自己探索，本指导不展开（提示：它本质也是一个工具，先检索本地文档再回答）。

其余优化方向见 `README.md` 的进阶一节：长期记忆改向量召回、工具调用失败自动重试等。优化部分只会影响加分。

9. 分数要求

总分 100 分：

模块	分值
任务 1 · 工具调用 <code>web_search</code>	30
任务 2 · 长期记忆	50
进阶 · RAG（本地知识库检索）	10
其余各类优化	10

判定方式基本就是前面教过的那几样——看仪表栏、查 `/api/capabilities`、读 `python main.py` 的启动行，照着自查就行。

9.1 任务 1 · 工具调用 `web_search`（30 分）

评分点	分值	怎么判定
工具能被启用	8	<code>is_available()</code> 返回 <code>True</code> ，且 <code>/api/capabilities</code> 的 <code>tools</code> 里能看到 <code>web_search</code>
<code>run()</code> 实现正确	14	问助手需要搜索的问题， <code>run()</code> 返回非空、且和问题相关的字符串
模型能正确调用	8	仪表栏出现 <code>web_search</code> 调用记录（CLI 则打印 <code>[调用工具 web_search ...]</code> ），说明 <code>description</code> 写得让模型分得清场景

9.2 任务 2 · 长期记忆（50 分）

评分点	分值	怎么判定
模块能被启用	5	模块顶部 <code>ENABLED = True</code> ，且 <code>/api/capabilities</code> 的 <code>long_term_enabled</code> 为 <code>true</code>
落盘正确（ <code>_load</code> / <code>_save</code> ）	12	<code>data/memory_store.json</code> 正常生成；按全局 <code>USER_ID</code> 存，且不会覆盖其他 user 的数据
<code>remember</code> 能抽取稳定事实	15	对话后 <code>facts</code> 列表新增了正确的事实，且做了去重（同一条不重复存）
<code>recall</code> 能召回并注入	12	相关问题能把对应事实召回到上下文；仪表栏「记住了什么」有显示
跨会话生效	6	A 窗口告诉它「我叫小明」，新建 B 窗口问「我叫什么」能答出来

9.3 进阶 · RAG（10 分）

不做不扣分，做了按下表加分。验收细则另见 `docs/rag-verification.md`。

评分点	分值	怎么判定
文档入库	3	跑完 <code>rag/ingest.py</code> ，向量库里有数据
检索工具被触发	4	问知识库相关问题，仪表栏出现 <code>knowledge_base</code> 调用与 <code>sources</code> 引用来源
回答基于检索结果	3	答案确实对应到检索到的文档； <code>docs/rag-verification.md</code> 清单能过

9.4 其余各类优化（10 分）

评分点	分值	怎么判定
至少落地一个优化方向	10	代码里能看到对应实现，且报告中有介绍 + 截图/对比

可选方向举例：长期记忆改向量召回、RAG 加重排序（rerank）或多路召回、工具调用失败自动重试、ReAct 反思自纠、接入真实网络搜索 API。多做几个不额外加分，把一个做扎实、在报告里讲清楚即可。

10. 提交要求

提交两部分：代码源码 和 大作业报告。

10.1 代码源码

- 交整个项目的源码，包含你实现的 `tools/web_search.py`、`memory/long_term.py`；做了进阶就把 `rag/`、`tools/knowledge_base.py` 等一并交上。
- **不要提交模型**：比如 RAG 用到的 embedding 模型权重，体积大且不属于你的代码，不要打包进去。
- 同样建议剔除：`.env`（里面有你的 key）、`__pycache__/`、虚拟环境目录、本地缓存等。`data/memory_store.json` 这类运行产物可留可不留。

10.2 大作业报告

报告里要有**各任务执行后的截图**，证明功能确实跑通了。仪表栏、`/api/capabilities` 都是好素材：

- 任务 1：助手调用 `web_search` 的截图（仪表栏的工具调用记录，或 CLI 打印）。
- 任务 2：跨会话记忆生效的截图——A 窗口存「我叫小明」、B 窗口取出来，以及仪表栏「记住了什么」。
- 进阶 RAG（若做）：检索命中、引用来源的截图。

如果做了优化，报告里还要有对优化的介绍：做了什么、怎么实现的、效果如何（最好配上优化前后的对比截图）。

提交渠道、截止时间、命名要求等待助教补充。